

Steganalysis: Technical Theory

Mathematical Foundations of Imperceptibility Metrics, Statistical Detection, and
Frequency-Domain Analysis

Contents

1	Introduction	2
2	Part 1: Imperceptibility Metrics	3
2.1	MSE — Mean Squared Error	3
2.2	PSNR — Peak Signal-to-Noise Ratio	3
2.3	SNR — Signal-to-Noise Ratio	3
2.4	SSIM — Structural Similarity Index	4
2.5	UQI — Universal Quality Index	5
2.6	NCC — Normalized Cross-Correlation	5
2.7	BER — Bit Error Rate	5
3	Part 2: Statistical Detection Methods (Classical LSB Steganalysis)	7
3.1	Why natural images have predictable statistics	7
3.2	Chi-Square Attack (Westfeld & Pfitzmann, 1999)	7
3.3	RS (Regular-Singular) Analysis (Fridrich, Goljan, Du, 2001)	8
4	Part 3: Information-Theoretic Measure	9
4.1	Shannon Entropy	9
5	Part 4: Frequency Domain Theory	10
5.1	The Fourier Transform, conceptually	10
5.2	Why analyze the FFT of the noise residual specifically	10
5.3	The radial energy split (low_freq_ratio / high_freq_ratio)	10
6	Part 5: Audio-Specific Theory (Planned Extension)	12
6.1	Why SSIM doesn't translate to 1D audio directly	12
6.2	Spectrogram-SSIM — the workaround	12
6.3	PESQ and STOI — perceptual audio quality metrics	12
7	Part 6: Video-Specific Theory (Planned Extension)	13
7.1	Why per-frame image metrics aren't sufficient alone	13
7.2	Inter-frame noise correlation	13
7.3	Optical flow / motion compensation	13
8	How These Theories Connect to the Verdict Engine	14

1 Introduction

This document covers the theoretical foundations behind every metric and method in the unified steganalysis pipeline (`steganalysis.py`) used across the LSB (Sem 6), CNN (Sem 7), and INR (Sem 8) phases of the project. For each method, it explains **what is measured, the underlying mathematics, why it works**, and **its limitations** — intended as both a reference for writing the results/methodology sections of the research paper and as viva preparation material.

The overarching design philosophy of the steganalysis pipeline is that **no single metric is sufficient**, because each one is blind to a different *type* of distortion. PSNR/MSE catch overall magnitude but miss structure. SSIM/UQI catch structure but can be insensitive to subtle, spatially-uniform noise. Chi-square/RS catch statistical signatures of specific embedding mechanisms (LSB) but say nothing about CNN/INR. Frequency analysis catches *where*, in frequency space, the distortion lives — which correlates with *what kind* of algorithm produced it. Entropy catches whether the information content of the signal increased, independent of where or how. A verdict requiring convergence of multiple independent signal types is more robust precisely because an embedding algorithm tuned to defeat any *one* test is unlikely to simultaneously defeat all of them.

2 Part 1: Imperceptibility Metrics

These all answer one question: *how different is the stego signal from the cover signal?* But they measure “different” in fundamentally different ways, which is why several are needed together.

2.1 MSE — Mean Squared Error

$$MSE = \frac{1}{N} \sum_i (cover_i - stego_i)^2$$

The simplest possible measure: average the squared difference across every sample (pixel, audio sample, etc.). Squaring does two things — it makes all differences positive, and it penalizes large errors disproportionately. An error of 10 contributes 100; an error of 2 contributes only 4.

Why squaring matters: if an embedding changes one pixel by 50 and a thousand pixels by 1 each, MSE is dominated by that single large change. This is both a strength (catches outlier distortions) and a weakness (a single bright artifact can make MSE look bad even if the image looks fine overall).

MSE has no inherent scale — its value depends on the dynamic range of the data (0–255 for 8-bit images vs ± 1.0 for normalized audio). This is the motivation for PSNR.

2.2 PSNR — Peak Signal-to-Noise Ratio

$$PSNR = 10 \cdot \log_{10} \left(\frac{MAX^2}{MSE} \right)$$

PSNR normalizes MSE against the maximum possible signal value (255 for 8-bit images), then converts to a logarithmic decibel scale. The log scale exists because human perception of signal quality is roughly logarithmic — the difference between “perfect” and “very good” matters more than the difference between “bad” and “very bad,” and dB compresses the latter while preserving resolution in the former.

Why it dominates steganography papers: it’s a single number, well-understood across signal processing, and decades of literature establish acceptable thresholds (>40 dB for images; audio is more perceptually forgiving in some bands, with >30 dB often cited).

Core weakness: PSNR treats every pixel/sample as equally important and assumes errors are perceptually uniform. Human vision is far more sensitive to changes in flat, smooth regions (a clear sky) than in busy textured regions (grass, static). Two images with identical PSNR can look completely different — one with error concentrated in a textured corner (invisible), the other in a smooth gradient (visible banding). This is the entire motivation for SSIM.

2.3 SNR — Signal-to-Noise Ratio

$$SNR = 10 \cdot \log_{10} \left(\frac{\text{signal power}}{\text{noise power}} \right) = 10 \cdot \log_{10} \left(\frac{\text{mean}(cover^2)}{\text{mean}((cover - stego)^2)} \right)$$

The key difference from PSNR: the denominator uses the **actual energy of the signal itself** as the reference, not a fixed maximum value.

Why this matters in practice: consider a dark image (pixel values near 0) vs a bright image (pixel values near 255), both with the same MSE from embedding. PSNR reports the *same* value for both, since $MAX = 255$ for both. SNR reports a *much worse* score for the dark image, because the embedding noise represents a much larger fraction of that image’s actual signal energy.

This makes SNR more representative of “how noticeable is this distortion relative to what’s actually there” — closer to how perception works (a small absolute change is very noticeable in a quiet/dark region, less so in a loud/bright one). In audio, SNR is the standard metric, because audio amplitude varies enormously (silence vs loud passages) and PSNR’s fixed-maximum reference doesn’t reflect this.

2.4 SSIM — Structural Similarity Index

$$SSIM(x, y) = [l(x, y)]^\alpha \cdot [c(x, y)]^\beta \cdot [s(x, y)]^\gamma$$

With $\alpha = \beta = \gamma = 1$ (standard formulation):

$$l(x, y) = \frac{2\mu_x\mu_y + C_1}{\mu_x^2 + \mu_y^2 + C_1} \quad (\text{luminance})$$

$$c(x, y) = \frac{2\sigma_x\sigma_y + C_2}{\sigma_x^2 + \sigma_y^2 + C_2} \quad (\text{contrast})$$

$$s(x, y) = \frac{\sigma_{xy} + C_3}{\sigma_x\sigma_y + C_3} \quad (\text{structure})$$

This is built on a fundamentally different philosophy than MSE/PSNR. Wang & Bovik’s insight (2004) was that **the human visual system is highly adapted to extract structural information from a scene**, and that perceived quality is closely related to whether structural information is preserved — not whether absolute pixel values match.

The three components, intuitively:

- **Luminance** compares the mean brightness of local windows — “is this region the same brightness?”
- **Contrast** compares the standard deviation — “does this region have the same amount of variation/texture?”
- **Structure** compares the normalized covariance (a correlation coefficient on mean-subtracted signals) — “do the patterns of light and dark line up in the same places, regardless of absolute brightness/contrast?”

The structure term is the clever one: by subtracting out the mean and dividing by standard deviation first, it measures *pattern correlation independent of overall brightness/contrast*. SSIM can therefore distinguish “the edges and textures are in the same places” (good, even if brightness shifted slightly) from “the edges and textures have moved or blurred” (bad, even if average brightness is identical).

Implementation detail: SSIM is computed over small sliding windows (typically 11×11 Gaussian-weighted) across the image, and the final score is the average of all local SSIM values. This locality means SSIM responds to *where* structural damage occurs, not just an aggregate average — closer to how a viewer scans an image.

Why C_1, C_2, C_3 exist: small constants that avoid division by zero in flat regions (where variance is 0) and stabilize the metric when means/variances are small.

Relationship to steganography: an embedding algorithm that adds noise uniformly (like LSB) tends to preserve SSIM well, because the noise has no structure — it doesn’t create or destroy edges, just adds tiny random jitter that the luminance/contrast/structure terms barely notice. An embedding that’s spatially correlated or concentrated (some deep-learning residuals) can lower SSIM even at the same PSNR, because it perturbs the *pattern*, not just the *value*.

2.5 UQI — Universal Quality Index

$$Q = \underbrace{\frac{\sigma_{xy}}{\sigma_x \sigma_y}}_{\text{correlation}} \cdot \underbrace{\frac{2\bar{x}\bar{y}}{\bar{x}^2 + \bar{y}^2}}_{\text{luminance distortion}} \cdot \underbrace{\frac{2\sigma_x \sigma_y}{\sigma_x^2 + \sigma_y^2}}_{\text{contrast distortion}}$$

UQI (Wang & Bovik, 2002) is the *predecessor* to SSIM — the same three-way decomposition (correlation/structure, luminance, contrast), without the stabilizing constants $C_1/C_2/C_3$, and as a single global statistic rather than a sliding-window local average.

Why it remains useful alongside SSIM: lacking stabilizing constants, UQI is more sensitive (sometimes oversensitive) to small distortions, and can swing wildly near zero-variance regions. Using both gives a “stable, perceptually-calibrated” reading (SSIM) and a “raw, mathematically pure” reading (UQI) of the same three distortion types. A large gap between UQI and SSIM can itself be informative — suggesting distortion is concentrated in flat/low-variance regions where UQI is unstable but SSIM’s constants smooth it out.

2.6 NCC — Normalized Cross-Correlation

$$NCC = \frac{\sum(\text{cover} \cdot \text{stego})}{\sum \text{cover}^2}$$

This comes from template-matching and watermarking literature, not general image quality. It asks: “*if the stego image is treated as a scaled copy of the cover, how good is that scaling relationship?*”

Why “normalized”: the denominator (sum of cover squared) normalizes out the cover’s own energy, so the result is a ratio rather than an absolute quantity. A value of exactly 1.0 means stego = cover pixel-for-pixel (in a least-squares sense). Values drifting from 1.0 indicate the stego image isn’t just “cover plus small noise” but has some systematic scaling or offset relationship to the cover.

Where most useful: watermark *robustness* testing — after an attack (compression, cropping, rotation), NCC between extracted and original watermark measures how much survived. In the steganalysis context, it is a secondary correlation check — a sanity metric catching gross structural problems (a stego image globally darkened, or with a constant offset added) that MSE/PSNR might not flag clearly.

2.7 BER — Bit Error Rate

$$BER = \frac{\text{number of differing bits}}{\text{total bits}}$$

The most literal possible metric: write every pixel value as 8 bits, XOR the cover and stego bit-patterns, count the 1s (differing bits), divide by total bits.

Why this is the “ground truth” metric for LSB specifically: LSB steganography operates *at the bit level* by definition — it replaces the least significant bit(s) of pixel values with payload bits. BER directly measures what fraction of those bits changed, a near-perfect proxy for “how much payload was embedded” if the embedding scheme is known. For a full 1-bit-LSB embed with random payload, roughly half the LSBs flip (a random bit has 50% chance of differing from the original), giving $BER \approx 1/8 = 0.125$ from that one bit-plane alone.

Why it is less meaningful for CNN/INR: these methods don’t operate on discrete bit-planes — they add continuous-valued residuals. A pixel might shift from 127 to 128 (changing the entire byte pattern in binary: 01111111 → 10000000, an 8-bit flip for a “1 unit” change!) purely

due to binary representation, not because 8 bits of payload were embedded. So BER for deep methods reflects *binary representation artifacts* of small numeric changes more than actual payload capacity — useful as a rough “how much did values shift” signal, but should not be interpreted the same way as in LSB.

3 Part 2: Statistical Detection Methods (Classical LSB Steganalysis)

These methods don't measure quality — they try to *detect the presence of hidden data* by exploiting the fact that embedding disturbs the natural statistics of a signal.

3.1 Why natural images have predictable statistics

A photograph of the real world has pixel values that aren't random — neighboring pixels tend to be similar (smooth gradients, correlated sensor noise), and the distribution of pixel *values* (the histogram) tends to follow smooth, slowly-varying curves rather than having sharp jumps between adjacent values. **Embedding a payload — especially a compressed or encrypted payload, which looks like random noise — disrupts these natural patterns in specific, mathematically characterizable ways.** Classical steganalysis exploits this disruption.

3.2 Chi-Square Attack (Westfeld & Pfitzmann, 1999)

The core observation: consider pixel values 100 and 101. In a natural image, these two values generally occur with *different* frequencies (e.g., value 100 might appear 500 times, value 101 480 times — close, but not identical, following the natural smooth histogram curve).

Now suppose **sequential LSB embedding** is performed with a payload that's effectively random bits (true of compressed/encrypted data). Flipping the LSB of a pixel toggles it between an even value and the next odd value — i.e., moves pixels *between* 100 and 101 (and similarly for every other adjacent pair: 102 <-> 103, 104 <-> 105, ...). With enough embedded random bits, the *combined* count of {100, 101} stays roughly the same (conservation — flipping doesn't create or destroy pixels, just moves them between the pair), but the values get redistributed *evenly between the pair* — because a random bit is equally likely to be 0 or 1.

The statistical test: for each pair $(2k, 2k + 1)$, compute the “expected” count under the null hypothesis of even redistribution:

$$expected = \frac{count_{2k} + count_{2k+1}}{2}$$

Then compute the standard Pearson chi-square goodness-of-fit statistic:

$$\chi^2 = \sum \frac{(observed - expected)^2}{expected}$$

This measures how far the observed histogram deviates from the “evenly split pairs” hypothesis. Convert χ^2 to a p-value using the chi-square distribution with degrees-of-freedom equal to the number of pairs tested.

Interpretation: if the stego image's histogram pairs are suspiciously close to perfectly even (low χ^2 , high p-value relative to what natural images show), that is the signature of embedding. The implementation treats $p < 0.05$ on the goodness-of-fit-to-even-pairs test as suspicious, corresponding to the pairs being unusually close to the 50/50 split that random-bit LSB embedding produces.

Critical limitation: this attack assumes **sequential** embedding starting from the first pixel. If the payload only fills part of the image (common in practice), only that region shows histogram disruption, and a global histogram test averages it out and may miss it. Historically the attack was extended to a sliding-window version testing increasingly large prefixes of the image to *locate* where embedding stops; the version in this pipeline is the simpler global test.

3.3 RS (Regular-Singular) Analysis (Fridrich, Goljan, Du, 2001)

More sophisticated than chi-square, and historically much harder to defeat, because it relies on **local smoothness** rather than histogram shape.

Step 1 — The discriminant function $f(x_1, x_2, x_3, x_4)$:

$$f(G) = \sum_i |x_{i+1} - x_i| \quad \text{for consecutive pixels in group } G$$

This is the “total variation” — the sum of absolute differences between adjacent pixels in a small group (here, 4 consecutive pixels). Natural images are locally smooth, so $f(G)$ tends to be small. Random noise (an embedded payload) increases local roughness, so $f(G)$ tends to increase.

Step 2 — The flipping function F_1 : flips the LSB of pixels at alternating positions within the group ($0 \leftrightarrow 1, 2 \leftrightarrow 3, 4 \leftrightarrow 5, \dots$). This simulates “what would this group look like with different LSB content.”

Step 3 — Classify each group as Regular (R), Singular (S), or Unchanged (U):

- **Regular (R):** $f(F_1(G)) > f(G)$ — flipping LSBs made the group *rougher* (what happens to a clean, smooth, natural group when its LSBs are perturbed)
- **Singular (S):** $f(F_1(G)) < f(G)$ — flipping LSBs made the group *smoother* (counter-intuitive; happens in noisy/already-perturbed groups)
- **Unchanged (U):** $f(F_1(G)) = f(G)$

Step 4 — Repeat with the “negative” flipping function F_{-1} (flips LSBs in the opposite pattern, or equivalently, flips all LSBs first then applies F_1), giving counts R^- and S^- .

The key theoretical result Fridrich proved: for a clean (unmodified) image, statistically $R \approx R^-$ and $S \approx S^-$ — the regular/singular group counts are roughly symmetric between the two flipping conventions, because there’s no embedded structure to break the symmetry. **LSB embedding breaks this symmetry in a predictable, quantifiable way** that depends on the fraction p of pixels that have been LSB-embedded.

Fridrich derived that $R(p), S(p), R^-(p), S^-(p)$ as functions of the embedding rate p are well-approximated by quadratic curves. Measuring R, S, R^-, S^- on the actual (possibly-stego) image allows solving a quadratic equation to estimate p — **the actual payload embedding rate**, not just yes/no:

$$2(R^- - R)p^2 + (R - R^- + S^- - S)p + (S^- - S) = 0$$

This quadratic comes from the theoretical mass-conservation and symmetry-breaking relationships Fridrich derived; solving it gives p directly.

Why RS survived longer than chi-square as an attack: it does not depend on global histogram shape at all — it depends on *local pixel correlations*, much harder for a steganography algorithm to preserve while embedding a meaningful payload. Adaptive/random-order LSB embedding defeats chi-square (it doesn’t create the simple sequential pattern), but RS still detects the *aggregate* statistical disruption regardless of embedding order, because it measures smoothness disruption, not positional pattern.

Why RS is computationally expensive: it requires examining every group of pixels and evaluating the discriminant function (and its flipped versions) for each — there is no FFT-style shortcut; it is an inherently per-group $O(1)$ operation repeated $H \times W/4$ times.

4 Part 3: Information-Theoretic Measure

4.1 Shannon Entropy

$$H(X) = - \sum_x p(x) \log_2 p(x)$$

Entropy measures the *average information content* (in bits) of a random variable — equivalently, how “spread out” or “unpredictable” a distribution is. For an 8-bit pixel channel, if every value 0–255 were equally likely, $H = \log_2(256) = 8$ bits (maximum entropy — completely unpredictable). If the image is a solid color, $H = 0$ (every pixel is the same value — zero information per pixel, completely predictable).

Natural images have entropy well below 8 — typically 6–7.5 bits per channel — because pixel values cluster around certain ranges (sky pixels cluster around blues, skin tones cluster in certain ranges) rather than being uniform.

Why embedding tends to increase entropy: a compressed or encrypted payload is, by design, close to maximally random (that is what compression/encryption achieves — removing redundancy). Mixing this near-random data into pixel values (via LSB or otherwise) pushes the pixel value distribution *toward uniform* — toward higher entropy. The histogram smooths out and becomes less peaked/structured.

Why the *delta* matters more than the absolute value: absolute entropy varies hugely between images (a busy textured photo naturally has higher entropy than a smooth gradient). For the *same* image before and after embedding, any entropy increase is attributable to the embedding process itself, making the delta a clean, image-independent signal — though for small payloads relative to image size, the delta can be tiny (fractions of a bit), requiring high precision.

5 Part 4: Frequency Domain Theory

5.1 The Fourier Transform, conceptually

Any signal (image row, audio waveform, 2D image) can be decomposed into a sum of sine/cosine waves of different frequencies, amplitudes, and phases — the Fourier Transform. For a 2D image, the 2D FFT decomposes the image into spatial frequencies: **low spatial frequencies represent slow, smooth variations** (broad gradients, overall shapes), and **high spatial frequencies represent rapid variations** (edges, fine texture, noise).

$$F(u, v) = \sum_x \sum_y f(x, y) \cdot e^{-2\pi i \left(\frac{ux}{M} + \frac{vy}{N} \right)}$$

The `fftshift` operation rearranges the output so the zero-frequency (DC) component is at the center of the array, making the radial low-to-high frequency structure visually intuitive (center = low frequency, edges = high frequency).

5.2 Why analyze the FFT of the noise residual specifically

noise = cover – stego isolates *just the change introduced by embedding*, with the original image content removed. The FFT of this isolated noise reveals the **frequency signature of the embedding algorithm itself**, separate from the frequency content of whatever image happens to be tested.

The physical intuition for why different embedding methods have different frequency signatures:

- **LSB embedding** with random payload bits, applied independently to each pixel, produces noise that is essentially **spatially white** — no correlation between neighboring pixels’ noise values. White noise has a *flat* frequency spectrum — equal energy at all frequencies — meaning a roughly uniform spread across low and high frequencies, but because there is so much of it relative to the smooth low-frequency content of real images, it tends to dominate the high-frequency bands *relative to what is naturally there*.
- **CNN-based embedding** — the encoder network has learned (via training) to add perturbations that are smooth/correlated, partly because convolutional kernels naturally produce spatially correlated outputs, and partly because training objectives (often including perceptual losses) explicitly penalize high-frequency artifacts that would be visible. This results in noise concentrated more in **low-to-mid frequencies**.
- **INR (coordinate network) embedding** — since the INR maps continuous coordinates to outputs via a smooth function (especially with SIREN’s sinusoidal activations, which are literally sums of sinusoids, directly frequency-interpretable), the residual can have a **more structured, potentially periodic frequency signature** tied to the network’s learned frequency components (ω_0 and the weight matrices set which frequencies the network can represent). This is genuinely the most “it depends on the trained model” case of the three.

5.3 The radial energy split (low_freq_ratio / high_freq_ratio)

The implementation draws a circle of radius = (image dimension / 2) / 4 around the center (DC) of the shifted FFT, and sums squared-magnitude energy inside (low) vs outside (high) this circle. Squaring the magnitude before summing relates to **Parseval’s theorem** — the sum of squared

FFT magnitudes relates to the total energy/power of the signal — so this ratio is literally asking “what fraction of the noise’s total energy lives in low vs high spatial frequencies.”

This single ratio is a crude but effective one-number summary of “is this noise smooth (low ratio) or sharp/random (high ratio).”

6 Part 5: Audio-Specific Theory (Planned Extension)

6.1 Why SSIM doesn't translate to 1D audio directly

SSIM's three components (luminance, contrast, structure) are defined over *local windows*, and the “structure” component specifically relies on comparing 2D spatial patterns — where edges/textures align. A 1D audio waveform doesn't have the same kind of 2D spatial structure; a sliding window over raw audio samples mostly captures amplitude envelope, not the rich texture information SSIM was designed around.

6.2 Spectrogram-SSIM — the workaround

A spectrogram is computed via the **Short-Time Fourier Transform (STFT)**: slide a window across the audio signal, take the FFT of each windowed segment, and stack these as columns to form a 2D image where one axis is time, the other is frequency, and the value is the magnitude (often log-scaled, giving dB) of each frequency component at that time.

Why this re-enables SSIM: a spectrogram *is* a 2D image — and critically, one whose structure (energy concentration across time and frequency) corresponds closely to *perceived* sound characteristics (pitch, timbre, rhythm). Two audio clips that “sound similar” tend to have visually similar spectrograms — harmonics, formants (for speech), and rhythmic energy bursts all show up as 2D structure. Computing image-SSIM on the cover's spectrogram vs the stego's spectrogram gives a perceptually-meaningful structural similarity score, reusing the same SSIM math.

6.3 PESQ and STOI — perceptual audio quality metrics

These are standardized (ITU-T recommendation, for PESQ) metrics specifically designed to *predict human listener quality ratings*, originally developed for telecom codec evaluation.

- **PESQ (Perceptual Evaluation of Speech Quality):** processes both signals through a model of human auditory perception (including time-alignment to handle delay, and a perceptual loudness/masking model based on how the cochlea responds to different frequencies), then computes a “disturbance” value mapped to a Mean Opinion Score (MOS)-like scale (roughly 1–4.5).
- **STOI (Short-Time Objective Intelligibility):** focuses specifically on *speech intelligibility* (can the words be understood?) rather than general quality — computes correlation between short-time envelope representations of clean and degraded speech across frequency bands.

Why these matter for audio steganalysis: PSNR/SNR on raw audio samples can be misleading because human hearing has **frequency-dependent sensitivity** (much more sensitive to mid frequencies, 1–4 kHz, than very low or very high frequencies) and **masking effects** (a loud sound makes nearby-in-frequency quiet sounds inaudible). An embedding that adds noise in a frequency/time region “masked” by the existing audio could have *terrible* raw SNR but be **completely inaudible** — and conversely, a tiny perturbation in a sensitive frequency band could be very noticeable despite excellent SNR. PESQ/STOI account for this; raw SNR does not. This is exactly analogous to why SSIM exists for images — PESQ/STOI are “the SSIM of audio,” but going further by modeling the actual auditory system.

7 Part 6: Video-Specific Theory (Planned Extension)

7.1 Why per-frame image metrics aren't sufficient alone

A video is not just “many independent images” — it has **temporal coherence**: consecutive frames are highly correlated (mostly the same content, small changes due to motion). Human perception of video quality is heavily influenced by **temporal artifacts** — flickering, inconsistent noise patterns between frames — that simply do not exist as a concept in single-frame metrics.

7.2 Inter-frame noise correlation

If the noise residual ($cover_{frame} - stego_{frame}$) is computed for each frame, and the correlation between residuals of consecutive frames is checked:

- **High correlation between frame residuals** suggests the embedding pattern is *static/fixed* relative to the frame (e.g., the same spatial pattern embedded in every frame, perhaps tied to a fixed key) — itself a detectable signature, since real-world camera/compression noise is typically *not* this consistent across frames.
- **Low correlation (independent noise per frame)** is closer to natural frame-to-frame variation, but if the *embedding* itself is independent per-frame, this could also flicker visibly — a temporal-domain analog of the spatial visibility problem.

7.3 Optical flow / motion compensation

Optical flow estimates, for each pixel, the apparent motion vector between consecutive frames — “where did this piece of the image move to in the next frame.”

Why this matters for steganalysis: without motion compensation, comparing frame N to frame $N+1$ directly conflates two different sources of “difference”: (1) genuine motion in the scene (a car moving, camera panning) and (2) embedding-induced noise. By first computing optical flow and *warping* frame $N+1$ back to align with frame N 's content, motion-induced differences can be subtracted out, isolating what remains — much closer to pure embedding noise (plus residual compression artifacts). This is the video analog of “isolating the noise residual” that the FFT-of-(cover-stego) approach performs for static images, except video requires this alignment step first for the residual to be meaningful.

8 How These Theories Connect to the Verdict Engine

The overarching design philosophy: **no single metric is sufficient because each one is blind to a different *type* of distortion.**

- PSNR/MSE catch overall magnitude but miss structure.
- SSIM/UQI catch structure but can be insensitive to subtle, spatially-uniform noise.
- Chi-square/RS catch *statistical* signatures of specific embedding mechanisms (LSB) but say nothing about CNN/INR.
- Frequency analysis catches *where*, in frequency space, the distortion lives — which correlates with *what kind* of algorithm produced it.
- Entropy catches whether the *information content* of the signal increased, independent of where or how.

A verdict that requires **convergence of multiple independent signal types** (rather than any single threshold) is more robust precisely because steganography research is an arms race — an embedding algorithm optimized to defeat any *one* of these tests (e.g., an adaptive LSB scheme tuned to pass chi-square) is much less likely to simultaneously defeat all of them, since they measure fundamentally different mathematical properties of the signal.